Type 3: Software Project

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

Zero Trust-Enabled SDN for Secure Load Balancing in Edge Networks

23CSE498 – Project Phase 2 (Panel Review 1) | **Team Number: 50**

S.No	Register Number	Student Name
1	CB.SC.U4CSE23262	K R Jeevan Raj
2	CB.SC.U4CSE23303	Aravind Kumar M
3	CB.SC.U4CSE23307	Arjun Krishnakumar
4	CB.SC.U4CSE23308	Ashwin Kumar RM
5	CB.SC.U4CSE23544	Bhuvanesh S

Guide: **Dr. Radhika N**, Professor, Department of CSE

September 9, 2026

Agenda

1. **Introduction** — the edge setting, and what we built
2. **Motivation** — why self-reported telemetry fails
3. **Literature Survey** — what exists, and the gap
4. **Problem Statement & Objectives**
5. **Architecture** of the proposed framework
6. **Solution** — the two rails and their enforcement
7. **Results** — a controlled two-arm comparison
8. **Limitations** and technical enrichment

The one sentence

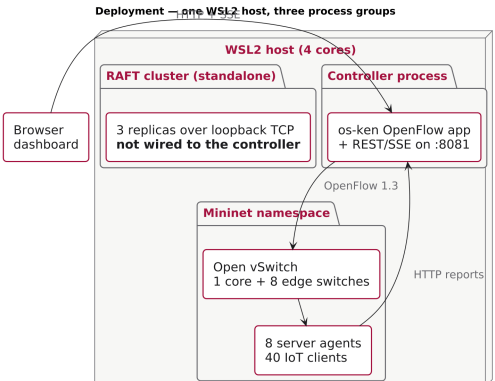
Every load balancer in production decides where to send work from what the servers *say about themselves*; we built an SDN control plane that decides from what it *observes*, revokes a misbehaving server in the data plane in under a second, and measured the difference against a complete no-zero-trust control arm.

1. Introduction: The Edge Setting

Edge computing pushes computation onto gateways, roadside units, factory-floor servers and campus racks. **Three facts collide there:**

1. **The perimeter model fails.** A node *inside* the network can be compromised, rented, misconfigured or simply selfish.
2. **Self-reported telemetry is the norm, and is gameable.** Kubernetes, cloud auto-scalers and MEC orchestrators all schedule on what a node says about itself. A node that under-reports load *attracts* traffic.
3. **Zero Trust is a doctrine, not a mechanism.** NIST SP 800-207 mandates continuous verification but specifies no way for a control plane to verify the *serving infrastructure* and act on the result in the data path.

Scope we are careful about: this is *demonstrated in an emulated network*, never "proven in production".



1. Introduction: What We Built

Rubric: UML Diagrams, DB Schema, APIs, Module Interaction & Deployment (5 Marks)

An SDN control plane that scores every edge server on **measured behaviour**, and when a server misbehaves **revokes it in the data plane** as real OpenFlow rules, while an append-only cryptographic ledger records why.

The system under test

- os-ken OpenFlow 1.3 controller application
- Northbound REST + SSE API on :8081
- Live dashboard, tamper-testable ledger ribbon

The test harness (not the product)

- Mininet + Open vSwitch topology
- Server agents doing *real bounded work*
- IoT clients and the attack simulator

Scale of one run

- 1 core + 8 edge OVS switches
- 8 edge servers, 40 IoT devices
- 6 armed attack classes, delayed onset
- ≈300 s per run, 4-core WSL2 box
- ≈27 000 recorded events per run

Why it is an experiment: we built a complete *no-zero-trust* version of the same system, launched by the same code. Most projects compare against a *description* of a baseline rather than a running one.

2. Motivation: The Attacker Does Not Need Cryptography

The core observation

Almost every real orchestrator schedules on what nodes *claim* about themselves. A node that under-reports its load therefore **attracts** traffic — which is exactly what an attacker wants. **The attacker does not have to break any cryptography to do this. It just has to lie in a field that everybody believes.**

Why the doctrine does not close it

NIST SP 800-207 says *never trust, always verify, continuously*. But commercial Zero-Trust products overwhelmingly verify **users and client devices at access time**. Verifying the **serving infrastructure itself, continuously, with consequences in the routing path**, is much less settled. That is the gap this project works in.

Where it bites in the real world

MEC / multi-tenant edge: an operator can host a tenant's workload on a node it does not fully control and still bound its exposure — traffic is revoked in under a second, with a signed record of why.

Operational technology: the scenario that keeps OT engineers awake is not a machine that stops — alarms fire for that — it is a controller that keeps "working" while lying about its state, because every safety decision downstream is computed from that state.

2. Motivation: Threat Model

Controller trusted; edge servers and IoT clients untrusted; adversaries non-adaptive

Adversary	Subject	Behaviour	Detection signal
Sybil / load-lying	edge server	under-reports load to attract work	CPU-honesty deviation
Blackhole	edge server	accepts then drops every task	timeout rate
Grayhole	edge server	drops a fraction (0.5) of tasks	elevated timeout rate
On-off	edge server	alternates good and bad, 20 s period	intermittent latency + timeouts
DDoS flood	IoT client	floods requests at $\approx 24\times$ normal rate	per-client request rate
Identity spoof	IoT client	claims another device's identity	source-IP pin mismatch
Bad credentials	IoT client	wrong PRESENT-80 key	challenge-response failure

In scope. All seven above, armed on a delayed onset so the system must detect a *change*, not a static configuration.

Out of scope, stated before being asked. Collusion, slow-and-low, adaptive attackers that learn our thresholds, and attacks on the controller itself.

3. Literature Survey: Five Threads

Thread	Key sources	What we took from it
Zero Trust & trust-aware edge	[1] NIST SP 800-207, [11] Ali 2023, [12] TREN 2025, [13] Huang 2024, [19] ETSI MEC	[1] is the doctrine we implement *and* the gap we claim. [11] is the closest prior work — same three ingredients — but stops at the access decision. [13] is the score-only design whose arithmetic ceiling our Finding 1 characterises.
Load balancing	[2] Mitzenmacher 2001, [14] Azar 1999, [3] Mitzenmacher 2000, [4] Dahlin 2000, [9] Jain 1984, [20] Little 1961	[2] is the source of our argmax → power-of-two-choices fix. [4] explains the one place we lose: stale least-loaded is worse than random. [9] is the fairness index we report.
Crypto, ledger & consensus	[5] PRESENT 2007, [18] ISO/IEC 29192-2, [6] Merkle 1988, [7] RAFT 2014	[5] is the admission cipher, checked against published test vectors. [7] gives the safety properties our RAFT tests are written against — crash-fault tolerant, *not* Byzantine.
SDN platform	[8] OpenFlow 2008, [10] Mininet 2010	[8] is the source of Finding 3: once a rule is installed, matching packets never return to the controller. [10] is why we say "emulated", never "production".
Learning & method	[15] UCB1 2002, [16] Wilcoxon 1945, [17] Holm 1979, [21] Nygard 2018	[15] is the online weight optimiser — a classical bandit, *not* deep RL. [16],[17] are the paired test and the multiple-comparison correction. [21]'s circuit breaker is our probation mechanism.

System	Reported headline	Enforcement
Ali, Gregory & Li (2023) [11]	F1 94.3 % anomaly detection; 96.1 % access-decision accuracy	access decision
TREN (2025) [12]	95 % detection accuracy; −20 % latency; +25 % throughput	policy enforcement
Huang et al. (2024) [13]	+13.37 to 21.87 % malicious-node detection rate	trust-based selection
This work	**97.9 % over 48 subjects** (8 classes); 8/8 detected; 0/40 honest mislabelled	**OpenFlow drop rules**

Read this as positioning, not as a benchmark

Cross-paper numbers in this area are **not comparable** and we do not present them as though they were. An "accuracy" over a two-class DDoS/Sybil problem and one over an eight-class problem with 40 honest subjects are not the same quantity.

The gap. The systems above report how well a trust mechanism *performs*. **None reports a limit of the trust mechanism itself.** A system reporting 95 % detection from a single trust rail cannot tell whether its remaining 5 % is noise or a structural blind spot — which is exactly what our Finding 1 identifies.

Research question

Can an SDN control plane continuously verify edge servers using *observed* rather than *claimed* behaviour, and enforce the verdict in the data plane fast enough to matter?

What has to be true for the answer to be yes

- Verification must run on evidence the node cannot forge — measured outcomes, not telemetry
- The verdict must reach the *data plane*, not just a scheduler's preference ordering
- It must be fast enough that an attacker's window is bounded
- The record must be tamper-evident after the fact

Our answer, at prototype scale

Yes — with isolation inside a 3-second bound — and with the important **negative result** that score-based trust alone is provably *insufficient* to do it.

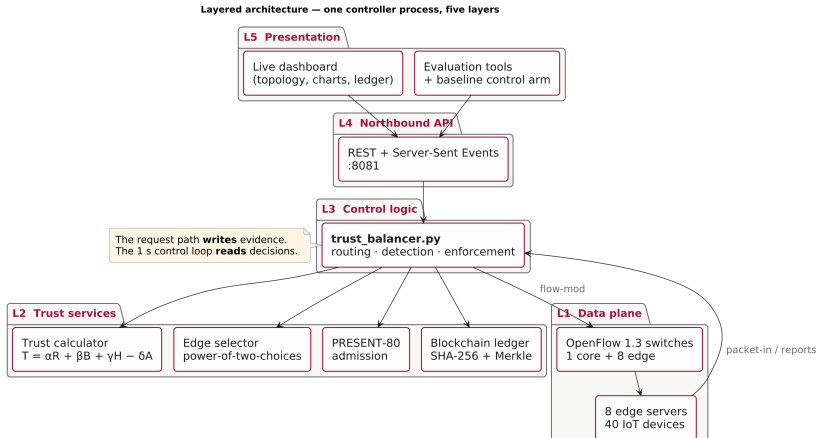
That negative result is the part we would most want carried forward, and it is developed on the next slides.

4. Objectives

1. **Characterise the insufficiency of score-based trust.** Derive the trust floor for a non-degrading liar analytically, verify it against the shipped calculator, and demonstrate it in a live run.
2. **Show the two rails are complementary, not redundant.** A trust score and an independent anomaly gate must each catch attackers the other misses.
3. **Enforce trust verdicts in the data plane with measured bounds —** OpenFlow drop rules and VIP rewrites, timed end to end.
4. **Compare against a running control arm**, not a description of one, plus a statistically grounded multi-baseline study with correction and effect sizes — *including the comparisons we lose*.
5. **Make the trust record tamper-evident** with forensic resolution, so an edit is localised rather than merely detected.
6. **Establish whether the routing weights can be learned online**, and report the result honestly either way.
7. **Separate selection strategy from hardware as the scaling limit —** determine what actually stops the fleet scaling.

5. Architecture: Five Layers, One Process

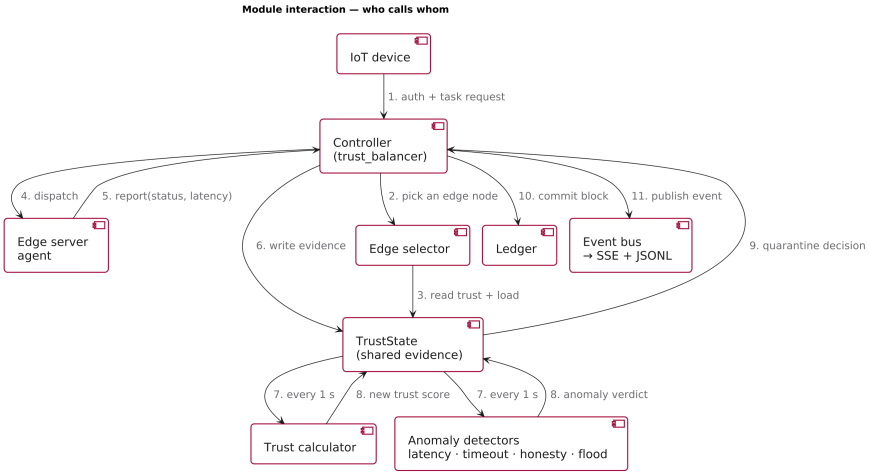
Rubric: UML Diagrams, DB Schema, APIs, Module Interaction & Deployment (5 Marks)



The trust engine and the ledger are libraries the controller calls in-process, not separate services. That is what lets the control arm remove enforcement without becoming a second codebase.

5. Architecture: Module Interaction

Rubric: UML Diagrams, DB Schema, APIs, Module Interaction & Deployment (5 Marks)



5. Architecture: API and Data Model

Rubric: UML Diagrams, DB Schema, APIs, Module Interaction & Deployment (5 Marks)

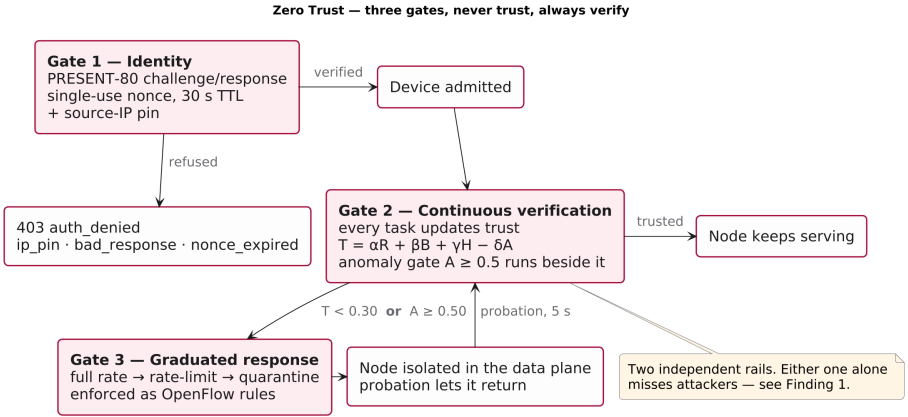
Method	Path	Request	Response
POST	/auth/challenge	{device_id}	{nonce}, 64-bit, TTL 30 s
POST	/auth/verify	{device_id, response}	{token} | 403 auth_denied{kind}
POST	/report	{status, latency_ms, ...}	task outcome; client_ip from the socket
POST	/register	{node_id, concurrency}	agent start-up registration
POST	/topology/links	{links:[...]}	link parameters the harness built
GET	/trust/score	?node_id=	{node_id: score} or all nodes
GET	/node/status	?node_id=	TrustState.snapshot()
GET	/ledger/verify	none	{valid, chain_length, batch}
GET	/api/events	none	Server-Sent Events stream

No relational database. The **ledger** is the store of record for trust, and its integrity is *cryptographic* — SHA-256 chain plus a Merkle root per block — rather than referential. The JSONL recording is the durable evidence log; configuration is YAML.

Why SSE and not WebSockets. It is one-way, plain HTTP, and needs no third-party library — which matters because the target machine is apt-only Python 3.14 with **no pip, no venv and no web framework**.

6. Solution: Zero Trust as Three Gates

Rubric: Software Architecture, Implementation, Integration & Tech Stack (5 Marks)



Measured at the gates in the live run: identity refused 3 devices (2 wrong-key, 1 spoof); continuous verification raised 291 anomalies; graduated response applied 39 quarantines and 21 re-steers. [data/comparison/comparison.txt]

The formula

$$T = \alpha\bar{R} + \beta\bar{B} + \gamma\bar{H} - \delta\bar{A}$$

α=0.35 reliability, β=0.25 behaviour, γ=0.25 honesty, δ=0.15 anomaly. EMA λ=0.85, initial trust 0.5, isolation threshold **0.30**.

Now consider a competent liar — a node that lies about its load but still serves every task correctly:

$$T = 0.35(1.0) + 0.25(0.96) + 0.25(0) - 0.15(1.0)$$

$$\mathrm{T} = 0.44 \;>\; 0.30$$

δ can only ever subtract 0.15. **Dishonesty alone can never push the score across the isolation line.**

Finding 1 — a characterised insufficiency

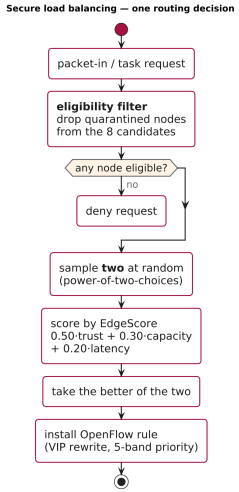
Score-based trust is **insufficient** for Zero Trust's continuous-verification requirement. An independent anomaly gate ($\bar{A} \geq 0.5 \Rightarrow$ quarantine regardless of T) is **necessary, not belt-and-braces**.

Confirmed live, not just on paper. In this run the sybil attacker srv3 ended at $T = 0.699$ — more than twice the isolation threshold — and was caught *only* by the anomaly gate. The blackhole srv6 was caught *only* by the trust score. **Neither rail alone catches both.** [comparison.txt]

Why this is a contribution, not a bug report. We kept the published formula byte-for-byte and added a documented gate beside it. Quietly retuning the weights until the demo worked would have destroyed the finding.

6. Solution: Secure Load Balancing

Rubric: Software Architecture, Implementation, Integration & Tech Stack (5 Marks)



Why power-of-two-choices, not argmax

Argmax is winner-take-all: every request goes to the single best node until its load catches up. We measured it starving the fleet as N grows.

At $N = 64$ servers	argmax	p2c
Packet delivery ratio	10.5 %	99.7–99.8 %
Tasks lost to timeout	79 114	0
Jain fairness	0.75–0.91	≈0.995

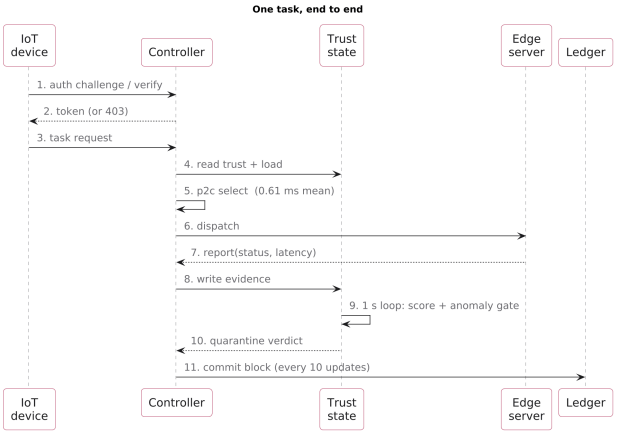
[data/scalability.csv]

The eligibility filter is the security join. Quarantined nodes are removed from the candidate set *before* scoring, so a trust verdict changes routing directly rather than only nudging a weight.

Honest caveat: with *no* attacker present we sit 10.7 % behind plain round-robin, because our 1 s load signal is stale relative to ≈0.2 s tasks — the regime Dahlin [4] describes.

6. Solution: One Task, End to End

Rubric: Backend, Frontend, Database & Core Modules Functional (8 Marks)



Enforcement is real: a quarantine verdict becomes an **OpenFlow drop rule**, and in-flight clients are re-dispatched in **18.13 ms mean**. Every decision is published exactly once to the event bus, so the dashboard and all 19 analysis tools read one stream. [data/nfr_report.txt]

6. Solution: Implementation Status

Rubric: Backend, Frontend, Database & Core Modules Functional (8 Marks)

Subsystem / Layer	Current implementation state	Status
Data plane & topology	1 core + 8 edge OVS switches, 8 edge servers, 40 IoT devices, VIP service model	Functional
SDN control plane	OpenFlow 1.3: proxy-ARP, per-connection VIP rewrite, 5-band priority table, meters	Functional
Trust engine	$T = \alpha R + \beta B + \gamma H - \delta A$, EMA λ=0.85, isolation 0.30, anomaly gate 0.50	Functional
Secure load balancing	EdgeScore + power-of-two-choices, eligibility filter, UCB1 online weight tuning	Functional
Security / admission	PRESENT-80 challenge-response, single-use nonce, source-IP pinning (roster + TOFU)	Functional
Attack detection	6 attacks, delayed onset; latency, timeout, honesty and flood tells; windowed classifier	Functional
Blockchain ledger	SHA-256 chain, Merkle root per block, batch of 10, in-browser re-verification	Functional
Northbound API	9 endpoints + SSE stream, standard library only	Functional
Dashboard (frontend)	Live topology, flow tables, 6 time-series charts, tamper-testable ledger ribbon	Functional
Evaluation & control arm	19 analysis tools; complete no-zero-trust control arm; scored comparison	Functional
Test suite	≈930 test functions across 52 files; CI on every push	Functional
RAFT consensus	Core + TCP transport + 3-replica live cluster, safety proven. *Not wired into the controller*	Partial
DDoS response	Detected and classified; throttling not built (needs a per-client-IP OpenFlow path)	Partial

7. Results: The Controlled Comparison

Baseline 2026-09-05 vs zero-trust 2026-09-08 — same topology, same 40 devices, same six attacks, same schedule

Metric	Baseline	Zero-trust
Packet delivery ratio	94.29 %	**99.43 %**
Mean per-device availability (honest)	85.67 %	**99.32 %**
Honest devices served below 50 %	5	**0**
Tasks lost to timeout	380	**32**
p95 task latency	260.0 ms	**147.8 ms**
Mean task latency	**83.7 ms**	92.4 ms
Throughput	20.74 task/s	20.61 task/s
Jain fairness, honest servers only	0.689	**0.998**
Requests routed to attacker nodes	30.5 %	**13.2 %**
Anomalies raised / acted on	502 / **0**	291 / **39**
Identity spoof (iot38 → iot1)	**SUCCEEDED** at 19.6 s	**REFUSED**
Wrong-key devices admitted	2	**0**
Trust blocks committed	— (no ledger)	643

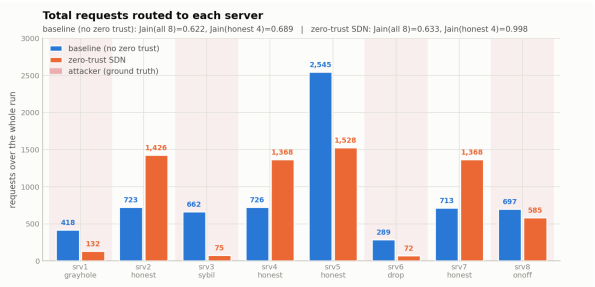
Containment, onset → isolation: sybil **7.3 s** · on-off **8.3 s** · blackhole **9.7 s** · grayhole **45.5 s**
The baseline never contains any of them, because it has no isolation path.

Mean latency is 8.7 ms *worse* in our arm — a small cost in the mean to remove the long tail: p95 nearly halves and timeouts fall 92 %. The grayhole is slowest to contain because a 50 % dropper must clear the timeout-rate threshold rather than fail outright.

[data/comparison/comparison.txt, 2026-09-08]

7. Results: Load Distribution and Fairness

Rubric: Backend, Frontend, Database & Core Modules Functional (8 Marks)



Jain over the *whole roster* barely moves (0.622 → 0.633) and hides the result; over the **four honest servers** it separates hard: **0.689 → 0.998**. Starving an attacker is correct behaviour, so it must not be scored as unfairness. [base_model/plot_load.py]

7. Results: Per-Server Trust, Both Arms

Rubric: Backend, Frontend, Database & Core Modules Functional (8 Marks)



Read per node, never as a fleet mean — averaging a blackhole's collapse against seven healthy servers produces a gentle dip and hides the finding. Note srv3 (sybil) stays high in our arm at $T = 0.699$: the score never condemns it, and the anomaly gate is what isolates it. That is Finding 1 visible in the data. [base_model/trust/]

7. Results: Detection Quality and NFR Validation

Rubric: Software Architecture, Implementation, Integration & Tech Stack (5 Marks)

Attack detection, 48 subjects

Measure	Result
Overall accuracy	47/48 = 97.9 %
Attacks detected at all	8/8
Placed in the right family	8/8
Exact label correct	7/8
Honest subjects mislabelled	0/40

The one miss: srv6's blackhole was labelled *grayhole* — the right family, one step too mild. Across five catalogued runs exact-label accuracy varies 5/8–8/8, so we do not quote a single best run. [evaluation/attack_report.py]

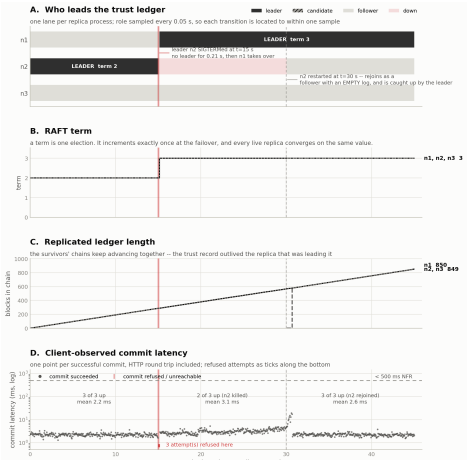
Non-functional requirements — all four PASS

NFR	Target	Measured	
Routing decision	<200 ms	0.61 ms mean, p95 0.83	PASS
Isolation re-dispatch	<3 s	18.13 ms mean, max 29.94	PASS
Blockchain overhead	<15 %	0.052 %, 643 blocks	PASS
RAFT commit	<500 ms	4.3–4.8 ms mean	PASS

Two caveats we state ourselves. The isolation figure is the *re-dispatch* time; *detection* is bounded by a 1 s poll with 3-poll persistence, so "<3 s" is met inclusively at best. And the RAFT row is measured on a standalone 3-replica cluster, not on the controller. [data/nfr_report.txt]

7. Results: Recorded RAFT Failover

3 replicas, real TCP, real SIGTERM — standalone cluster, not the live controller



What the recording shows

- 853 commits attempted, **850 succeeded** (99.6 %)
- Latency mean **2.6 ms**, median 2.4, max 19.4
- Leader `n2` killed by SIGTERM at $t = 15.0$ s
- `n1` elected after **0.21 s**; service gap 0.21 s
- 3 attempts refused while unreachable — and refusing is correct
- Degraded phase (2 of 3 up) costs only 2.23 → 3.08 ms

On restart `n2` rejoins with an *empty* log: this is crash-fault tolerance with an in-memory log, not Byzantine fault tolerance and not durable storage. We state that before being asked. [data/raft_timeline.jsonl]

8. Limitations and What Comes Next

Known limitations, stated plainly

- **RAFT is built, tested and demonstrated — but not wired into the controller.** All 643 blocks of the live run carry `raft_term = 0`.
- **DDoS is detection-only.** Throttling needs a per-client-IP OpenFlow path; we deferred it rather than shipping half of it.
- **One live run per configuration**, so no confidence intervals on the live figures.
- Detection latency is an **upper bound**, measured from controller start.
- Scale past 8 servers is a **queueing simulation**, not a live network; all runs are localhost, not multi-host.

Next steps, in order

1. **Wire RAFT into the commit path.** Three blockers remain: nobody drives the tick; `commit()` would block an OpenFlow handler thread for up to 2 s; a `None` return is currently discarded silently.
2. **Build the per-client-IP throttle** so the flood detector has an enforcement path, closing the DDoS story.
3. **Repeat the live runs with multiple seeds** to put intervals on the headline table.
4. **Improve grayhole containment** — 45.5 s is our weakest result, and a fractional dropper needs a sharper tell than timeout rate alone.

9. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	K R Jeevan Raj	Network Virtualization and Software-Defined Networking Board Infinity	Coursera	12 Hrs
2	Aravind Kumar M	Software Defined Networking The University of Chicago	Coursera	**30 Hrs**
3	Arjun Krishnakumar	Cisco ACI Training: Design and Implement Data Center SDN	Udemy	10 Hrs
4	Ashwin Kumar RM	Network Security Cisco Learning and Certifications	Coursera	12 Hrs
5	Bhuvanesh S	Modern Network Security 2026: Zero Trust, SASE, Cloud & SOC	Udemy	10 Hrs

Requirement: each student completes one approved 10–20 hour online certification from a recognised platform, relevant to the project.

All five map onto parts of the system their holders own: *Network Virtualization* and *Software Defined Networking* cover the SDN substrate and the OpenFlow control plane, *Cisco ACI* covers data-centre SDN and the policy fabric, *Network Security* covers the admission and detection path, and *Zero Trust / SASE / SOC* covers the doctrine the whole project implements.

References I: Zero Trust and Load Balancing I

Numbering matches Appendix B of the project report

Zero Trust and trust-aware edge computing

[1] S. Rose, O. Borchert, S. Mitchell and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, National Institute of Standards and Technology, Aug. 2020.
→ The doctrine we implement, and the gap we claim. It mandates continuous verification but specifies no mechanism for verifying the *serving infrastructure*.

[11] B. Ali, M. A. Gregory and S. Li, "Trust-aware task load balancing in multi-access edge computing based on blockchain and a zero trust security capability framework," *Trans. Emerging Telecommunications Technologies*, vol. 34, no. 12, e4845, 2023. DOI 10.1002/ett.4845.
→ The closest prior work, with the same three ingredients of zero trust, blockchain and MEC load balancing. We differ in enforcing the verdict in the data plane and in measuring against a control arm.

[12] "Securing edge based smart city networks with software defined networking and zero trust architecture (TREN)," *J. Network and Computer Applications*, 2025. DOI 10.1016/j.jnca.2025.104318.
→ The smart-city and SDN application domain.

[13] M. Huang, Z. Li, F. Xiao, S. Long and A. Liu, "Trust Mechanism-Based Multi-Tier Computing System for Service-Oriented Edge-Cloud Networks," *IEEE Trans. Dependable and Secure Computing*, vol. 21, no. 4, pp. 1639-1651, 2024. DOI 10.1109/TDSC.2023.3285927.
→ Trust scoring for edge task placement. This is the score-only design whose arithmetic ceiling our Finding 1 characterises.

References I: Zero Trust and Load Balancing II

Numbering matches Appendix B of the project report

[19] ETSI GS MEC 003, "Multi-access Edge Computing (MEC); Framework and Reference Architecture," V3.1.1, 2022.
→ The deployment mapping: controller to MEC orchestrator, edge servers to MEC hosts, IoT clients to UE.

Load balancing: why we replaced argmax with power-of-two-choices

[2] M. Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing," *IEEE Trans. Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094-1104, 2001. DOI 10.1109/71.963420.
→ The source of our fix. Sampling two candidates and taking the better one gives far better tail behaviour than picking the single best.

[14] Y. Azar, A. Z. Broder, A. R. Karlin and E. Upfal, "Balanced Allocations," *SIAM J. Computing*, vol. 29, no. 1, pp. 180-200, 1999.
→ The originating balanced-allocations result behind [2].

[3] M. Mitzenmacher, "How Useful Is Old Information?," *IEEE Trans. Parallel and Distributed Systems*, vol. 11, no. 1, pp. 6-20, 2000. DOI 10.1109/71.824633.
→ Herd behaviour caused by stale load information. Our poll interval is 1 s while tasks finish in about 0.2 s, which is the regime this paper describes.

[4] M. Dahlin, "Interpreting Stale Load Information," *IEEE Trans. Parallel and Distributed Systems*, vol. 11, no. 10, pp. 1033-1047, 2000. DOI 10.1109/71.888643.
→ **This explains the one place we lose.** Stale least-loaded performs worse than random, which is why we sit 10.7 % behind round-robin in the no-attack case.

[9] R. Jain, D.-M. Chiu and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," *DEC Research Report TR-301*, 1984.
→ The fairness index reported in Section 7 (0.689 to 0.998, honest servers only).

[20] J. D. C. Little, "A Proof for the Queuing Formula $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383-387, 1961.
→ The diagnostic that proved a live run's occupancy was fabricated rather than real.

Lightweight cryptography and the tamper-evident ledger

[5] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin and C. Vikkelsoe, "PRESENT: An Ultra-Lightweight Block Cipher," in *Cryptographic Hardware and Embedded Systems (CHES 2007)*, LNCS 4727, pp. 450-466, 2007.
→ The cipher behind device admission, and the published test vectors our implementation is checked against.

[18] ISO/IEC 29192-2:2019, *Information security, lightweight cryptography, Part 2: Block ciphers*.
→ The standardisation that makes PRESENT-80 a defensible choice for constrained devices, and the framing for its 80-bit key being below modern margins.

[6] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology (CRYPTO '87)*, LNCS 293, pp. 369-378, 1988.
→ The Merkle tree giving per-record proofs. Our `_hash_pair` was fixed to positional concatenation so that proofs are order-sensitive, as the construction requires.

[7] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proc. USENIX Annual Technical Conf.*, pp. 305-319, 2014.
→ RAFT. Safety properties R-01 to R-18 are tested against this paper's statements. It is crash-fault tolerant and *not* Byzantine, a distinction we state before being asked.

SDN platform

[8] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69-74, 2008.
→ The protocol our decisions are installed as, and the source of Finding 3: once a rule is installed, matching packets never return to the controller.

[10] B. Lantz, B. Heller and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010.
→ Mininet, the emulation testbed. It is why we say "demonstrated in an emulated network" and never "proven in production".

Learning and statistical method

[15] P. Auer, N. Cesa-Bianchi and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, no. 2/3, pp. 235-256, 2002.
→ UCB1, the online EdgeScore weight optimiser. A classical bandit and *not* deep reinforcement learning, because weight tuning is an exploration problem when no labelled data exists at runtime.

[16] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80-83, 1945.
→ The paired signed-rank test behind every p-value we report. Chosen over a paired t-test because our outcomes are bounded rates, skewed and zero-inflated.

[17] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian J. Statistics*, vol. 6, no. 2, pp. 65-70, 1979.
→ The multiple-comparison correction. One system against four baselines is four tests, so the family-wise false-positive rate is about 19 % rather than 5 %. Holm is uniformly more powerful than Bonferroni for the same guarantee.

[21] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018.
→ The circuit-breaker pattern. Our *probation* mechanism is its half-open leg, and it is what stopped quarantine being an absorbing state.

Thank You

Questions welcome — including the ones about what we have not finished.

Backup: Design Patterns and Engineering Decisions

Rubric: Software Architecture, Implementation, Integration & Tech Stack (5 Marks)

Pattern	Where	Justification
Layered architecture	L1 to L5	The trust engine must be callable with enforcement removed — that is what makes the control arm a controlled experiment rather than a second codebase
Strategy	`edge_selector`, `static_router`	Routing policy is a config key (`selection: p2c`), so the two arms differ by a key, not a fork
Protocol / structural typing	`CommitBackend`, `Transport`	`RaftBackend` is a drop-in for `LocalLedgerBackend`; `RaftNode` did not change at all to gain a TCP transport
Observer / pub-sub	`EventBus` → SSE + JSONL	Every decision is published exactly once; dashboard and analysis tools read one stream
Pure function	`attack_classifier`	No I/O, no locks, no state, so live and offline verdicts come from one implementation
Decorator	`TimingCommitBackend`	Commit cost is measured without the ledger knowing it is being timed
Multi-armed bandit (UCB1)	`ai_optimizer`	Weight tuning is an exploration problem rather than a training problem, because no labelled data exists at runtime

Stack: Python 3.14 · os-ken (Ryu fork) · Mininet + Open vSwitch, OpenFlow 1.3 · stdlib `ThreadingHTTPServer` + SSE · PRESENT-80 · SHA-256/Merkle · pytest.

Backup: Data Integrity, Concurrency and Error Handling

Concern	Mechanism
Ledger integrity	SHA-256 chain + Merkle root per block; the browser recomputes the hash itself rather than trusting the controller's `valid` flag
Deterministic replication	Log entries carry only content; `index`, `previous_hash` and `merkle_root` are recomputed per replica at apply time — the fix for a non-deterministic genesis block and a stale ledger snapshot taken at proposal time
Concurrency	One `RLock` guards `RaftNode`/`Ledger` (re-entrant, because `apply_fn` re-enters it); a second lock serialises whole `commit()` calls; `commit()` releases the first while waiting, or the driver thread deadlocks
Trust-state consistency	All mutation under one lock; the invariant `sum(_inflight) == len(_dispatches)` is asserted, after a register leak once fabricated load on honest nodes
Replay resistance	Nonces are single-use (popped on first verification) and expire after 30 s
Identity integrity	The fleet key is shared, so PRESENT-80 authenticates possession of the key rather than the device. That gap is closed by source-IP pinning against a provisioned roster, with TOFU as the fallback
Error handling	Structured `AuthError(kind)`; a 403 also *publishes* an event, so a refusal is visible in the recording; meters degrade to allow/quarantine where unsupported, logged once

Backup: The Statistical Study and Its Negative Results

600 paired seeded runs — 5 strategies × 4 scenarios × 30 seeds. This is a **simulation harness driving the real selector and calculator**, not a Mininet run — a different instrument from the live comparison in Section 7.

- 14 of 16 comparisons favour the system, $p_{\mathrm{adj}} = 7.45 \times 10^{-9}$, rank-biserial $r = +1.00$
- SLO-violation reduction vs the trust-ablated variant: sybil −15.1 %, drop −25.3 %, both −29.8 %
- Failure-rate reduction −97.7 %; malicious-traffic share −97.5 to −97.7 %

The results that did not go our way, reported because they are ours

- **We lose twice, both with no attacker present:** −10.7 % vs round-robin and −4.8 % vs random. Cause isolated by a poll-interval sweep: 1 s load-signal staleness, exactly the effect Dahlin [4] describes. It disappears at a 0.01 s poll.
- **A capacity cliff at $N = 4$ servers with 2 attackers —** with half the fleet hostile the system loses the SLO metric. It is fine from $N = 6$.
- **The Random-Forest warm start did not beat static weights** (2 of 4 scenarios significant, and *against* it). We report UCB1's online tuning as the working mechanism and the RF path as a measured dead end.
- **Isolation is 3.0 s median, 4.0 s max in the harness —** the "<3 s" target is met inclusively at best, by construction of the 1 s poll.